

Architecture Decision Records

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-17
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Record binding architectural decisions, their context, alternatives, consequences and reconsideration triggers.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 ADR policy

Each ADR is immutable after approval except for status and superseding references. New information produces a new ADR. Status values: Proposed, Accepted, Superseded, Rejected. All entries below are **Accepted** for the v1 baseline on 2026-08-05.

2 ADR-001 - Build an Android application, not a PWA

Context: The defining feature is a reliable scheduled alert that can wake a locked phone, provide native notification actions and recover after process death. Browser background execution and autoplay constraints cannot guarantee clock-like behavior.

Decision: Deliver a signed Android APK. Reuse web skills through React Native rather than treating a PWA as the production alarm surface.

Alternatives: PWA; Trusted Web Activity; Capacitor wrapper. These remain suitable for a lightweight companion but not the reliability core.

Consequences: Native Android permissions, lifecycle, testing and signing are required. Portfolio value increases through native integration. iOS is not automatically supported.

Reconsider when: Browser standards and OS policy provide a verified equivalent exact-alarm/full-screen mechanism across killed processes.

3 ADR-002 - Android-only version 1

Context: Android and iOS have materially different background scheduling and full-screen presentation capabilities. Designing both before proving the product would dilute reliability.

Decision: Scope v1 to Android phones/foldables, with responsive tablet UI. No shared promise of identical iOS behavior.

Consequences: Faster focused implementation and accurate platform claims. Architecture keeps domain logic portable where practical.

Reconsider when: Android v1 has real usage evidence and an iOS capability study defines honest fallbacks.

4 ADR-003 - React Native UI with a Kotlin reliability core

Context: React Native is efficient for forms, navigation and portfolio development, but due alarm actions cannot depend on JavaScript cold startup.

Decision: Use React Native 0.86.x/New Architecture for everyday UI and Kotlin for scheduling, receivers, services, Room repositories, notification actions, locked alarm activity and resilient player.

Alternatives: Pure Kotlin/Compose; all-JavaScript alarm plugin. Pure Kotlin maximizes uniformity but reduces the creator's preferred workflow. JS-only creates an unacceptable stop-action dependency.

Consequences: Two-language codebase and typed bridge tests. Alarm path survives RN failure.

Reconsider when: Project maintainers choose full Compose and can migrate without backup/behavior regression.

5 ADR-004 - Room is the logical source of truth

Context: Alarm receivers need data before React loads, and dual JavaScript/native stores create divergence.

Decision: Room stores reminders, schedules, profiles, occurrences, operations and active session. React receives DTOs through native use cases. DataStore holds only small preferences/device envelope.

Consequences: Native migration discipline is required. JS state is view/cache state, not durable truth.

6 ADR-005 - Schedule only the globally earliest due occurrence

Context: Registering every recurring occurrence increases wake/synchronization complexity and makes changes harder to reconcile.

Decision: Persist derived occurrences and register one ordinary next one-shot AlarmManager event. Recalculate after every relevant mutation/event. Do not use repeating alarms.

Consequences: Minimal OS alarm footprint and clear battery story. Scheduler correctness becomes central and receives extensive tests.

Reconsider when: Platform constraints demonstrate a reliability regression at supported scale that cannot be solved by reconciliation.

7 ADR-006 - Use user-authorized exact alarms with transparent Limited mode

Context: Exact reminders are core, but modern Android requires special access and exact alarms have battery cost.

Decision: Request `SCHEDULE_EXACT_ALARM` contextually. Prefer `setAlarmClock()` for enabled exact user-visible occurrences. When denied, offer explicit inexact Limited mode or keep reminder Needs setup. Do not silently degrade.

Alternatives: `USE_EXACT_ALARM`; inexact-only. `USE_EXACT_ALARM` is more restricted and distribution-sensitive; inexact-only breaks the intended alarm experience.

Consequences: Health and permission education are required. Access may be revoked and exact alarms canceled, requiring reconciliation.

8 ADR-007 - No persistent idle service or time polling

Context: Battery efficiency is a defining requirement.

Decision: No service remains active while idle; no JS interval observes the clock; no periodic worker delivers reminders. A bounded foreground service exists only while an audible alert is active.

Consequences: Android system scheduling carries idle timing. Service and wake-lock lifetime become measurable invariants.

9 ADR-008 - Locked full-screen; unlocked system notification; no overlay

Context: A full-screen alarm is useful when the user is not interacting with the device but irritating when they are. Android owns heads-up notifications and restricts full-screen intents.

Decision: At trigger, native code checks interactive/lock state. Locked or non-interactive may use FSI/native alarm if eligible. Unlocked uses a high-importance notification without FSI. Foreground app also shows an in-app strip. Never request `SYSTEM_ALERT_WINDOW`.

Consequences: The app cannot guarantee a precise 20% dropdown height. OEM/channel settings may suppress heads-up; notification shade is fallback.

10 ADR-009 - Media plays only after explicit Play

Context: The user's desired ritual is to acknowledge the reminder and then view/listen. Autoplay can be intrusive, disclose private content and conflict with platform audio rules.

Decision: Alarm tone/vibration may alert according to profile; attached media begins only after Play. Play stops the alarm before Media3 playback.

Consequences: Clear separation between alarm audio and reminder content; notification action must launch playback reliably.

11 ADR-010 - Copy imported media into app-specific storage

Context: Linking only to a gallery URI can break when permission expires, file moves or original is deleted.

Decision: Stream selected files into app-owned private storage, hash and validate, then release source access. Internal filenames are UUIDs.

Consequences: Uses additional device storage and uninstall removes the copy. Library is reliable and backup is deterministic.

12 ADR-011 - Use system pickers, not broad gallery permission

Context: User needs selected files, not persistent read access to the entire gallery.

Decision: Use Photo Picker for visual media and Storage Access Framework for documents/audio/export/import. Do not request broad media read permission in v1.

Consequences: Provider behavior varies and must be streamed safely. Privacy/permission surface is smaller.

13 ADR-012 - Use Media3 and native playback activity

Context: Accepted playback should remain functional if RN navigation is unavailable and should use supported Android media infrastructure.

Decision: Use AndroidX Media3 in a native activity for due playback; RN may use the same native component for previews through an abstraction.

Consequences: Codec behavior is device-dependent; unsupported media is handled explicitly. Adds native library size.

14 ADR-013 - Logical JSON/media backup, never raw database copy

Context: Raw SQLite couples archives to schema/filesystem and can include transient state.

Decision: Export versioned DTO JSON, media bytes, manifest and checksums. Import maps through repositories/migrations.

Consequences: More code but stable portability, conflict handling and validation. Database implementation can evolve.

15 ADR-014 - Plain ZIP in v1; encryption later

Context: A portable manual archive is required immediately. Correct encrypted archive design adds key derivation, recovery and compatibility complexity.

Decision: V1 writes a standard plaintext ZIP with explicit privacy warning. No developer-held key. P1 may introduce an authenticated passphrase format as a new archive version.

Consequences: User must protect the file. Portability and recoverability are high. Marketing must not call it encrypted.

16 ADR-015 - No Internet permission and exclude Android automatic backup

Context: "Local-only" can be undermined by accidental SDK traffic or operating-system cloud backup.

Decision: Production manifest has no Internet permission/analytics. Data extraction rules and legacy backup rules exclude all app data from system cloud backup and device transfer. Manual ZIP is the migration path.

Consequences: No in-app update check, remote crash reporting or cloud sync. Users transfer archives themselves.

17 ADR-016 - Room/file outbox and operation journals bridge atomicity gaps

Context: Room cannot atomically commit with AlarmManager or filesystem/document providers.

Decision: Persist desired scheduler generations and pending file/restore operation phases. Reconciliation completes or rolls back after crashes.

Consequences: More explicit state and repair code, but no false cross-system transaction assumption.

18 ADR-017 - Minimal direct-boot alarm envelope

Context: Credential-encrypted Room/media may be unavailable before first unlock after reboot.

Decision: Mirror only opaque next ID/time/profile class into device-protected storage. Reschedule a generic alert after locked boot; require unlock for personal details/media. Full reconciliation follows normal boot/unlock.

Consequences: Improved reboot reliability without leaking labels. Adds envelope consistency tests.

19 ADR-018 - Three reusable default profiles

Context: Configuring every sound/vibration/timeout repeatedly increases complexity.

Decision: Seed Gentle, Standard and Persistent profiles. Reminders reference profiles. Profile edits show affected count. Persistent is explicitly “not for emergencies.”

Consequences: Simpler authoring and consistent channels. Users need clear distinction between app preference and channel-effective behavior.

20 ADR-019 - Android SDK and React Native baseline

Context: The project starts on 2026-08-05 and should use an actively supported framework while meeting current distribution rules.

Decision: React Native 0.86.x stable line; minSdk 26; compileSdk 37; initial targetSdk 36; retest API 37 and move target when policy/dependency readiness requires.

Consequences: Covers modern Android while retaining API 26. Version facts are refreshed in MR-22 before implementation/release.

21 ADR-020 - Apache License 2.0

Context: The project is portfolio-worthy and may benefit from contributors while preserving patent/license clarity.

Decision: Application source uses Apache-2.0. Brand assets, screenshots and user media have separate provenance. Contributions require sign-off under repository policy.

Consequences: Commercial reuse is allowed under license terms; no warranty. Repository must include NOTICE/third-party license records as applicable.

22 ADR-021 - No medical, emergency or guaranteed timing claim

Context: Platform-controlled alarms can be disabled/delayed, and the app lacks regulated validation.

Decision: Product is a personal media reminder. Copy and portfolio materials explicitly exclude medical/emergency reliance and absolute delivery guarantees.

Consequences: Persistent profile remains bounded and cannot be marketed as critical alert infrastructure.

23 Supersession procedure

A new ADR states which prior ADR it supersedes, migration effect, backup/permission impact, release threshold and rollback. Code that contradicts an Accepted ADR is not merged merely because tests pass; the decision must be changed transparently first.

23.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.